Procesor pe 8-biti

Arhitectura multi-cicle implementata in Verilog

Author: Denis-Cătălin Bratosin

Universitatea Transilvania

Facultatea de Inginerie Electrica si Stiinta Calculatoarelor

1 Introducere

Acest proiect este incercarea mea de a modela folosind Verilog un procesor pe 8 biti. Procesorul suporta urmatoarele operatii

- Operatii aritmetice: ADD, SUB
- Operatii logice: AND, OR
- Shiftari: SHIFL, SHIFR
- Operatii la memorie: LOAD, STORE

Acesta a fost conceput cu scop didactic pentru a intelege hands-on functionarea unui CPU.

Arhitectura acestuia este multi-cycle in conditiile in care pentru instructiunile LOAD/STORE se executa intr-un singur tact de ceas iar celelate se executa in doua tacte.
Pe viitor pot fi adaugate cu siguranta mai multe operatii.

2 Arhitectura Procesorului

Procesorul este compus din 5 componente principale:

- **ALU** Executa operatiile aritmetice si logice
- **Control Unit:** Interpreteaza instructiunile si genereaza semnale de control pentru celelalte componente
- **Registre:** Prezinta 8 registre interne din care ALU isi va prelua operanzii si ulterior va stoca rezultatul operatiei
- **External Memory:** Memorie cu un singur port de R/W pentru a incarca in registrii sau a salva din acestia in memorie
- **Modulul superior de CPU:** Modulul in care sunt integrate si legate toate aceste componente

3 Format instructiuni

Instructiunile sunt sub forma de cuvinte scrise pe 32 de biti sub forma de hexa-zecimal preferabil pentru intelegerea mai bine a acestora.Mai multe informatii in capitolul lor propriu-zis.
Instructiunile pot fi divizate in cele de citire/scriere si cele aritmetice/logice.

Table 1: Aritmetice/Logice

Bits 31-24	**Bits 23-16**	**Bits 15-8**	**Bits 7-0**
Opcode	Destinație	Adresă/Operand1	Adresă/Operand2

Table 2: Format instrucțiuni pentru LOAD/STORE

Bits 31-24	**Bits 23-16**	**Bits 15-8**	**Bits 7-0**
Opcode	Destinație	Sursa	Nefolosit

4 ALU

```
    module alu(
    input          clk,
    input        rst_n,

    input [7:0]  data1,
    input [7:0]  data2,
    input [2:0] select,
    output reg[7:0]    out
);
```

Acestea sunt semnalele modulului pentru ALU. Acesta are 2 intrari pentru operanzi, una pentru selectia operatiei si o iesire pentru rezultat. Operanzii se vor prelua din cele 8 registre interne si rezultatul se va stoca tot acolo.

select	**opp**	**descriere**
001	ADD	data1 + data2
010	SUB	data1 - data2
100	AND	data1 & data2
101	OR	data1 | data2
110	SHIFL	data1 << data2
111	SHIFR	data1 >> data2

5 Registre

```
    module regfile(
    input   clk,
    input rst_n,
    input    we,//Write enable

    input[2:0] write_address,
    input[7:0] write_data,//Port de write din memoria mare
    input[7:0] write_data_alu,//port de write din alu
    input      sel_alu,

    input[2:0] read_addr1,
    input[2:0] read_addr2,

    output reg[7:0] out1,
    output reg[7:0] out2

);
```

Acestea sunt semnalele modulului bancului de registre. Aici avem mai multe semnale dar o sa le explicam pe fiecare in parte.

In cazul scrierii in registru:

- `we` - write enable pentru registre
- `write_address` - adresa registrului la care vrem sa scriem
- `write_data` - datele pe care vrem sa le scriem (in cazul in care LOAD din memorie in registru)
- `write_data_alu` - datele pe care vrem sa le scriem (in cazul in care avem un rezultat al unei operatii realizate de ALU)
- `sel_alu` - selectie pentru cand scriem in registru din ALU sau din memorie

In cazul citirii:

- `read_addr` - adresa de la care sa citim
- `out` - iesire pentru date

Bancul de registrii are 2 porturi de iesire si unul singur de intrare. Doua porturi de iesire ce sunt legate la ALU iar iesirea acestuia intra inapoi in registru. Scrierea este sincrona dar citirea este asincrona.

```
case(sel_alu)
 1'b1: if(we) reg_file[write_address] <= write_data_alu;
 1'b0: if(we) reg_file[write_address] <= write_data;
endcase
```

Selectia de unde se vor citi datele din ALU sau din memorie.

6 Memorie

```
    module memorie_1rw #(
    parameter ADDR_WIDTH = 8 ,
    parameter MEM_DEPTH = 256 , // MEM_DEPTH <= 2^ADDR_WIDTH
    parameter WORD_BYTES = 8 // latimea casutei )(
    //Memorie read-write cu 1 port 256 memorii a cate 8 biti fiecare
    input clk,
    input rst_n,

    input ce,//chip enable
    input we,//write enable

    input [ADDR_WIDTH-1:0] address,
    input [WORD_BYTES-1:0] write_data,

    output[WORD_BYTES-1:0] read_data

);
```

Memoria este similara registrelor avand scrierea sincrona si citirea asincrona. Semnalele fiind similare, folosim **ce** pentru enable la memorie pentru a citi si aditional **we** pentru a scrie.

7 Unitatea de control

```
    module control_unit(
    input clk,
    input rst_n,

    input[31:0] instruction,

    output reg[7:0] address_m,//adresa memorie
    output reg[2:0] address_r,//adresa registre interne

    output reg we,//Write enable pentru registre
```

```
    //Write si chip enable pentru memorie
    output reg mem_we,
    output reg mem_ce,

    //Alu
    output reg[2:0] source1,
    output reg[2:0] source2,
    output reg[2:0] select,
    output reg sel_alu

);
```

Unitatea de control are rolul de a primit instructiunea(cuvant pe 32 de biti) si a trimite semnale de control celorlalte componente.

7.1 address_m si address_r

Primule semnale despre care vom vorbi mai in detaliu sunt address_m si address_r, acesta sunt folosite in cazul in care avem de a face cu o intructiune de LOAD/STORE in rest acesta sunt 0.

```
always @(posedge clk or negedge rst_n)
if(~rst_n) address_m <= 'd0 ; else
case(instruction[31:24])
 8'h10:   address_m <= instruction[15:8];
 8'h11:   address_m <= instruction[23:16]; //Daca e store destinatia e in memorie
 default: address_m <= 'd0;
endcase
=====================================================
 8'h10:   address_r <= instruction[23:16];
 8'h11:   address_r <= instruction[15:8];

 !! Acele 2 linii fac parte dintr-un always mai mare
```

- In cazul unui LOAD address_m va reprezenta sursa de la care vom citi(adresa din memoria externa) iar address_r va fi destinatia(adresa registrului unde vom incarca valoarea citita la adresa respectiva din memorie).
- In cazul unui STORE acestea vor fi inversate si va fi nevoie sa le preluam de pe alti biti.
- In celalte cazul ascestea vor fi 0.

7.2 we, mem_we si mem_ce

Acestea sunt semnalele pentru enable,write enable ale memoriei si ale registrelor.

- we - este semnalul de write enable al registrelor
- mem_we - semnal de write enable al memoriei
- mem_ce - semnal de chip enable pentru memorie

7.3 source1, source2, select, sel_alu

```
always @(posedge clk or negedge rst_n)
if(~rst_n)             source1 <=               'd0; else
if(instruction != 'd0) source1 <= instruction[15:8]; else
                       source1 <=               'd0;

always @(posedge clk or negedge rst_n)
if(~rst_n)             source2 <=              'd0; else
if(instruction != 'd0) source2 <= instruction[7:0]; else
                       source2 <=              'd0;
```

- source1 si source2 - reprezinta adresele operanzilor din registre in cadrul instructiunilor aritmetice/logice.
- select - reprezinta operatia ce trebuie executata, in cazul in care avem LOAD/STORE acesta va fi 0
- sel_alu - acesta va fi 0 doar in cazul de LOAD/STORE, indica faptul ca se va salva in registru ceea ce este iesire de la alu

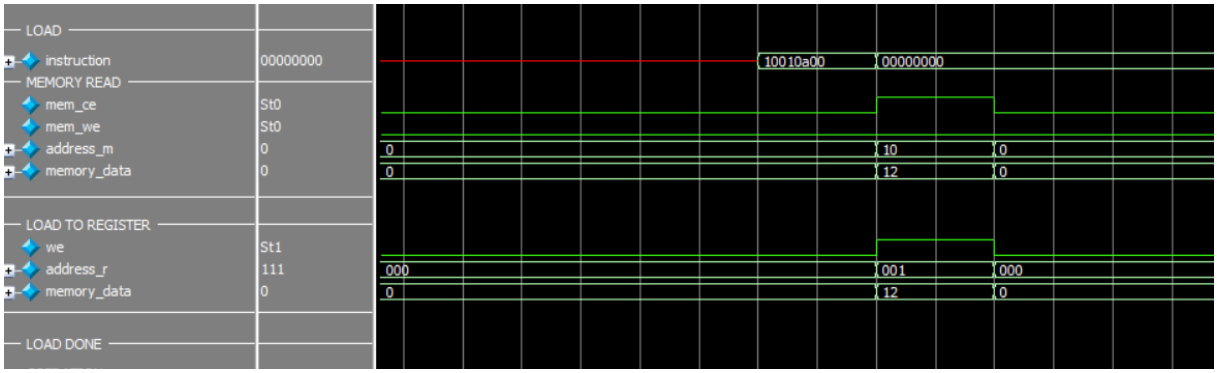
8 Exemplu de functionare

In scop de simulare avem artificial valorile:

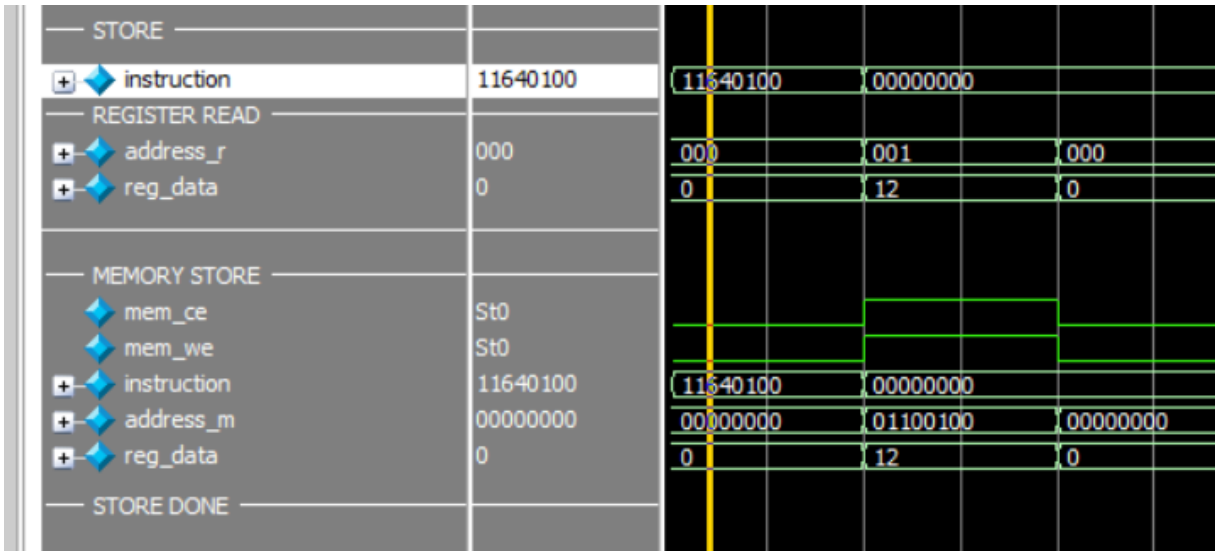
- Memory[10]=12;
- Reg[4]=15;
- Reg[5]=9;

O sa simulam pas cu pas aceste instructiuni folosind ModelSim:

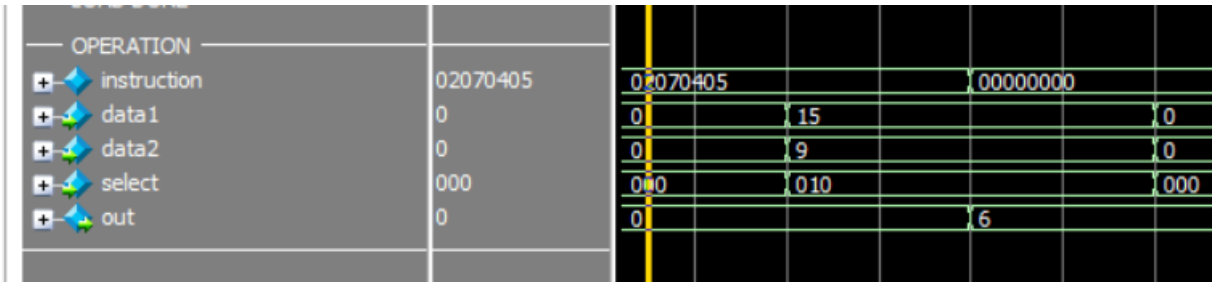
- **0x10010A00** LOAD in Reg[1] din Memory[10]
- **0x11640100** STORE din Reg[1] in Memory[100]
- **0x02070405** SUB in Reg[7] dintre Reg[4] si Reg[5]



În partea de sus sunt semnalele implicate în citirea din memorie: pe `address_m` avem adresa 10 (0x0A), de unde se citește valoarea 12. În partea de jos sunt semnalele implicate în scrierea în registru pentru finalizarea instrucțiunii LOAD: la registrul 1 (0x01) se scrie valoarea 12 citită anterior.



În prima etapă, se citește din registrul 1 (0x01) valoarea 12. Ulterior, pentru scrierea în memorie se activează semnalele `ce` și `we`, se pune adresa pe `address_m`(0x64 sau 100 in baza 10) și se transferă valoarea citită prin `reg_data`. Acești pași finalizează operația de STORE.



Pentru o operație aritmetică sau logică, instrucțiunea necesită doua tacte de ceas:
- Primul tact este folosit pentru a citi datele din registre și a le aduce în ALU.
- Al doilea tact este utilizat pentru efectuarea efectivă a calculului în ALU.

Iesirea out este legata la intrarea write_alu_data a bancului de registre. Acesta va stii sa aleaga sa scrie ce este pe aceasta intrare datorita semnalului sel_alu.

9 CPU

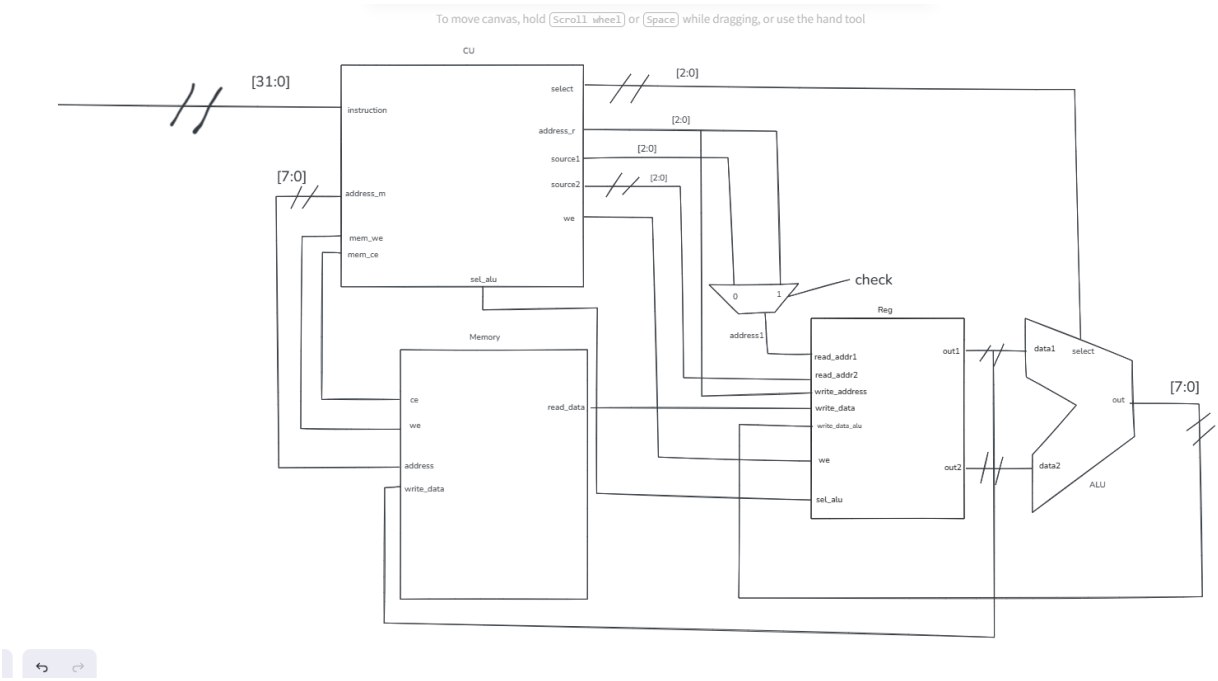


Figure 1: Schema naiva

Asa va arata procesorul dupa ce vom lega toate componentele intre ele. Singurul lucru ce va trebuii explicat va fi multiplexorul 2:1 dintre **address_r** si **source1**.
In cazul unui STORE vom avea prima oara de citit din registru valorea de la address_r care o vom pune pe read_addr1 si vom lua out1 si pune pe write_data al memoriei. In cazul unei